Distributed Caching

spread the cache over many nodes, and grow it safely

Watch the 4-min video

In one line: a distributed cache partitions keys across nodes so memory and throughput scale; use consistent hashing with virtual nodes (or hash slots) so adding a node moves only about 1/N of keys, route in the client, a proxy or the servers, give each shard a replica, grow gradually with warm-up, and keep the database protected while hit ratios recover.

1 From three nodes to thirty

Your three cache nodes are full and the hit ratio is dropping. You need thirty. Add them the wrong way and almost every key moves, the cache goes cold, and the database gets crushed.

A **distributed cache** splits the key space across many nodes, so total memory and throughput grow with the node count. Each key lives on one node, chosen by a **partitioning function**. The questions are: which function, and who runs it?

2 Which function: modulo vs consistent hashing

Naive `hash(key) mod N` works until N changes: going from 3 nodes to 4 remaps about **three quarters** of all keys. Consistent hashing (lesson 5) fixes it: nodes and keys sit on a ring, each key belongs to the next node clockwise, and adding a node moves only about **1/N** of the keys. Virtual nodes keep the load even.

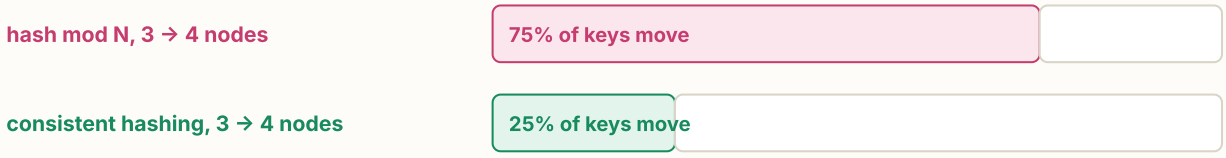


Figure 1. Growing from 3 to 4 nodes: modulo hashing moves about 75% of keys; consistent hashing about 25%, the new node's fair share.

3 Who routes: client, proxy or server

Client-side sharding: the cache client library knows the node list and hashes keys itself, like Memcached clients with ketama. Fast, no extra hop, but every client must share the same node list and algorithm. **Proxy-based**: clients talk to a proxy (twemproxy, Envoy's Redis proxy, mcrouter) that routes keys; clients stay simple at the cost of a hop. **Server-side**: Redis Cluster puts routing in the servers, with hash slots and MOVED redirects (lesson 28).

Client-side	Proxy	Server-side
library hashes keys	twemproxy, Envoy Redis	Redis Cluster slots
(ketama)	proxy routes keys	MOVED redirects
+ no extra hop	+ simple clients	+ resharding built in
− same list in every client	− extra hop, proxy HA	− cluster-aware clients

Figure 2. Three places to put the routing logic, with their trade-offs.

4 When a node fails

When a cache node dies, its share of keys becomes misses: with 10 nodes, about **10%** of keys. The database sees a sudden jump in load, so it needs headroom, or protection (rate limits, load shedding, request coalescing). **Replication** helps most: each shard has a replica that takes over with its data, so the hit ratio barely dips.

WORKED EXAMPLE: SCALING FROM 3 TO 30 NODES

Don't jump at once. With consistent hashing and virtual nodes, add nodes **gradually**, a few at a time: 3 → 6 → 10 → 15 → 20 → 25 → 30. Each step moves only a fraction of keys (3 → 6 moves half, but later steps move less: 25 → 30 moves about 17%), and the database absorbs the extra misses before the next step. Optionally, **pre-warm** new nodes by copying hot keys before shifting traffic to them. With Redis Cluster, migrate hash slots in small batches.

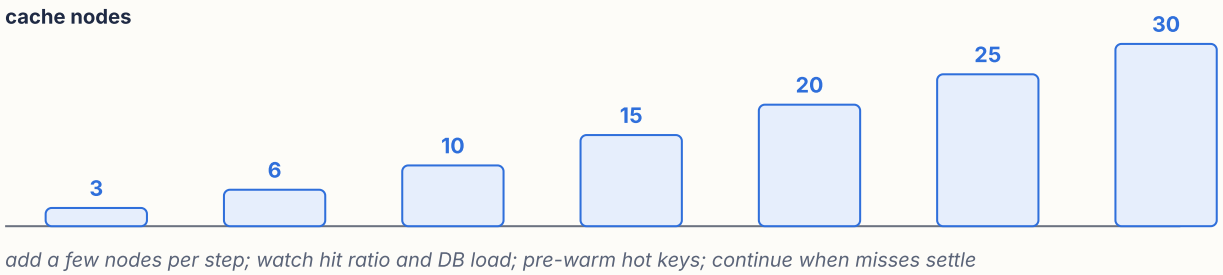


Figure 3. Grow in steps, letting the hit ratio recover between them.

5 Operating tips

- Watch the **hit ratio** and **database load** during every step.
- Keep TTLs **jittered**, so warmed keys don't all expire together (lesson 19).
- A bigger cluster means more connections from every app server: 200 pods × 30 nodes × a pool of 10 = 60,000 connections. Use pooling, multiplexing clients, or a proxy.
- Make sure every client agrees on the node list; a client with an old list reads and writes the wrong nodes.

COMMON MISTAKES

- Modulo sharding "for now", then a cold cache at the first resize.
- Doubling the cluster in one step during peak traffic.
- No replicas, so every node failure is a burst of database load.

6 Key terms

Partitioning function: maps a key to a node

Proxy sharding: a proxy picks the node (twemproxy, mcrouter)

Pre-warming: copying hot keys to new nodes before traffic

Client-side sharding: the library picks the node

Hash slots: Redis Cluster's 16,384 buckets

Shard replica: a copy of one node's data for failover

7 Interview questions

Q1. How do you add a cache node without a mass of misses? BASIC

Use consistent hashing with virtual nodes, or slot-based sharding, so only about 1/N of the keys move, add nodes gradually, pre-warm hot keys, and keep the database protected while hit ratios recover.

Q2. What happens when a node fails? BASIC

Its keys turn into misses until rebuilt, so use replicas for failover and give the database headroom.

Q3. Client-side sharding or a proxy for a polyglot company with 15 services? SCENARIO

A proxy (or Redis Cluster) avoids keeping 15 client libraries in sync on node lists and hashing; client-side is fine for one language and gives the lowest latency.

Q4. How many keys move when you go from 10 to 11 nodes with consistent hashing? SCENARIO

About 1/11, roughly 9%: the new node's share. With modulo, about 10/11, roughly 91%.

Q5. Two app versions use different node lists during a deploy. What breaks? TRICKY

They hash the same key to different nodes: one writes or invalidates on a node the other never reads, so stale data and extra misses appear. Roll out node-list changes atomically or via a proxy.

Q6. Why is connection count a scaling concern for distributed caches? TRICKY

Every app instance keeps connections to every node; it multiplies with pods and nodes and can exhaust file descriptors or memory on the cache servers. Pool, multiplex or use a proxy.

Next: SD-F-017 · CDN and HTTP Caching backendroadmap.com